

Introduction to Digital Twins for Software Engineers

Learning objectives

By the end of this chapter you will be able to:

1. **Classify** a proposed system as a digital model, a digital shadow, or a digital twin, by tracing the direction of its data flows.
2. **Identify**, in a running system, the three pieces of state every digital twin holds: what it sensed, what it believes, and what it commanded.
3. **Predict** how a twin misbehaves once its model stops matching the physical object.
4. **Derive** a check that detects that divergence from sensor readings alone.

Scope and prerequisites

One worked example runs from start to finish: a potted plant, a soil-moisture sensor, and a pump that waters it. Through it, the chapter covers the model/shadow/twin distinction, the anatomy of a twin, and the way twins fail as their models age. It does not cover co-simulation, modelling formalisms, or any specific commercial platform, and it is explanation rather than a build guide — to type code and watch a twin run, work through `digital-twin-plant-moisture-tutorial.md` alongside it.

You are assumed to write software professionally and to be comfortable with state, scheduled jobs, and testing. You are not assumed to know control theory, or ever to have wired up a sensor.

1. Why an engineer should care

You already know how to build a service that reads a sensor and draws a chart. A digital twin is the harder thing that chart is often mistaken for.

The distinction has teeth because the twin *acts*. A dashboard that is wrong produces a confused operator. A twin that is wrong waters the plant at 3 a.m. until the pot overflows. Bidirectional interaction between the physical object and its virtual counterpart is the centre of the concept, not an optional extra [1]. The discipline is also unsettled: practitioners report that the techniques for building and maintaining twins are still not well understood [2]. You are not arriving late to a solved problem.

Four terms, defined once and used consistently from here on. The **physical twin** is the real object: this pot, this plant. The **digital twin** is the software that mirrors it. A **sensor** reports the physical twin's state to the software; an **actuator** changes it on the software's command. The pump is an actuator.

2. Three systems, one plant

Kritzinger and colleagues separate three arrangements by how much of the data flow is automatic [3]. Take them in order — the sequence is the lesson.

A digital model. You keep a spreadsheet holding a drying-rate estimate for your plant. Once a week you poke the soil, type in a number, and the sheet predicts when to water

next. No data moves on its own in either direction; a person carries every reading in and every decision out.

A digital shadow. Now you push a moisture probe into the pot. It posts a reading every fifteen minutes to a service that stores it and plots it. Data now flows automatically from pot to software — but only that way. When the chart dips, *you* fill the watering can. Most systems sold as digital twins stop here.

A digital twin. The same service now opens a valve when it decides the soil is too dry. Data flows automatically in both directions, and the loop closes without you. That closure is the whole difference, and notice what it costs: a wrong shadow annoys you, while a wrong twin has a pump.

So the first question to ask about any system claiming to be a twin is not what it models. It is: **what does it change, and who approved that change?**

3. Anatomy: what the twin actually holds

Let $m(t)$ be the soil moisture at hour t , as a percentage of the pot’s capacity, and let r be the drying rate in percentage points per hour. The twin’s model is one line:

$$m(t + 1) = m(t) - r$$

Suppose the probe reads $m(0) = 60$, the twin’s stored estimate is $r = 2.0$, and the plant wilts below 30. The twin projects forward:

$$t_{\text{dry}} = \frac{60 - 30}{2.0} = 15 \text{ hours}$$

Three distinct pieces of state are now in play, and confusing any two of them is the classic beginner’s bug:

- **Sensed state** — 60. What the probe said. Noisy, and already stale by the time it arrives.
- **Modelled state** — the estimate $r = 2.0$ and the projection to 15 hours. What the twin *believes*. This exists nowhere in the physical world.
- **Commanded state** — “open the valve for 8 seconds at hour 15”. What the twin decided to do about the gap.

A dashboard has only the first. A twin has all three, and must reconcile them every cycle, because the physical twin does not know what the software believes.

4. How this breaks

The plant grows. Summer arrives. The real drying rate climbs to $r = 3.0$, and the soil crosses 30 at hour 10. The twin, still holding $r = 2.0$, waters at hour 15. The plant sat wilting for five hours while the software reported that all was well — and the twin’s logs show no error at all. Nothing crashed. The model aged out from under the plant, and nothing in the system was built to notice.

This gap between a twin’s model and its physical twin is why serious work in this area treats **online model updating** — refitting a model from data collected while the system runs — as a core requirement rather than a refinement [4].

Work the detection out one step at a time. The twin predicted $m(1) = 58$; the probe reported 57. One point of disagreement is sensor noise, not evidence. By hour six, though, the twin predicts 48 and the probe says 42. **The residual — prediction minus measurement — is growing in one direction.** That signature is what you watch for: noise cancels out over time, while a one-sided bias means the model itself is wrong.

The remaining step is yours (Exercise 2): decide what the twin should do about it.

5. Exercises

Exercise 1 (objective 1). Your neighbour’s system texts them “water me”, and they do. Classify it, then state the smallest change that would make it a digital twin. *Hint: count the automatic flows and their directions.*

Exercise 2 (objectives 3, 4). Continuing §4: the twin has six hours of one-sided residuals. Write down a rule that re-estimates r from the last N readings. Then the harder half — what happens if N is small and a spilled cup of tea enters the window? *Hint: the rule must not trust one reading more than it trusts the model.*

Exercise 3 (objective 2). The valve sticks open. Describe how sensed, modelled and commanded state now disagree, and design the check that catches it. *Hint: after watering, the twin makes a prediction it can verify.*

6. Summary and where to go next

A digital twin is not a model, and not a dashboard. It is a model wired to a physical object in both directions, so that it senses, believes, and acts (§2, §3). Its characteristic failure is not a crash but silent divergence: the model ages, the object changes, and the twin keeps acting confidently on a stale belief (§4).

For the practices around building one, Talasila and colleagues work through what realising a twin from real assets and models demands [5], and Gil and colleagues compare open-source frameworks worth building on rather than reinventing [6]. The companion tutorial builds this plant twin in about seventy lines of Python.

7. References

- [1] *Foundational Research Gaps and Future Directions for Digital Twins*. Washington, D.C.: National Academies Press, 2024. doi: 10.17226/26894.
- [2] L. Cleophas *et al.*, “A community-sourced view on engineering digital twins: A report from the EDT.Community,” in *Proceedings of the 25th International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings*, in MODELS ’22. New York, NY, USA: Association for Computing Machinery, Nov. 2022, pp. 481–485. doi: 10.1145/3550356.3561549.

- [3] W. Kritzinger, M. Karner, G. Traar, J. Henjes, and W. Sihn, “Digital Twin in manufacturing: A categorical literature review and classification,” *Ifac-PapersOnline*, vol. 51, no. 11, pp. 1016–1022, 2018, Accessed: Dec. 11, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2405896318316021>
- [4] A. Thelen *et al.*, “A comprehensive review of digital twin — part 1: Modeling and twinning enabling technologies,” *Structural and Multidisciplinary Optimization*, vol. 65, no. 12, p. 354, Nov. 2022, doi: 10.1007/s00158-022-03425-4.
- [5] P. Talasila, P. H. Mikkelsen, S. Gil, and P. G. Larsen, “Realising Digital Twins,” in *The Engineering of Digital Twins*, J. Fitzgerald, C. Gomes, and P. G. Larsen, Eds., Cham: Springer International Publishing, 2024, pp. 225–256. doi: 10.1007/978-3-031-66719-0_11.
- [6] S. Gil, P. H. Mikkelsen, C. Gomes, and P. G. Larsen, “Survey on open-source digital twin frameworks—A case study approach,” *Software: Practice and Experience*, vol. 54, no. 6, pp. 929–960, Jun. 2024, doi: 10.1002/spe.3305.